Understanding of the new Readme file

Here is how the plan's phases line up with the seven modules in your Implementation.pdf.

Module 1: Data Ingestion & Graph Construction

- **P1, parsing and cell library:** this replaces the PyVerilog/Yosys step. The netlists are already gate-level, so a strict custom parser is faster and more controllable. It also builds the Trojan labels from instance names and log.txt.
- **P2, multi-view graphs:** this builds the structural and functional graphs and stores them as PyG Data objects. The temporal view is only a placeholder here and gets filled in during P3 and P4.

Module 2: Feature Engineering

- **P3, logic simulation:** the simulator produces activation probability, toggle rate and entropy, since no traces ship with the data.
- **P4, feature matrix:** it builds the node features: gate type, in/out degree, distance to outputs, and the simulation features. The CodeBERT embedding from NeuroGuard-X is implemented but off by default.

Support phases (no module of their own)

- **P5, splits and metrics:** this sets up leave-one-family-out splits and metrics such as AUPRC. It runs before any model, so the evaluation can't be shaped by results.
- **Section 3A, training protocol:** early stopping, per-epoch logging, checkpoints and training reports. It applies to Module 3 and later to Module 7.

Module 3: Multi-View GNN Ensemble

- **P6:** GCN, GraphSAGE and GAT, with weighted-voting and transformer fusion. It outputs the node suspicion scores and the graph-level probability, and includes the mandatory ablation against a single GraphSAGE model.

Module 4: Context Expansion

- **P7:** top-k suspicious nodes, 2-hop expansion, and the anonymized JSON, extended with gate-role hints and a netlist snippet.

Modules 5 and 6: LLM Reasoning, plus Bias & Consistency Control

- **P8 covers both:** I merged them because they share one evaluation harness.

- Module 5 is the structured verdict, explanation and fix.
- Module 6 is the A/B swap and the cross-model or cross-temperature checks.
- It also adds the baselines (including argmax(gnn_score)) and the confidence calibration.

Module 7: Feedback Loop

- **P9:** soft labels from consistent LLM verdicts and the contrastive alignment, tested in three arms to check the “less LLM reliance” claim. The optional Yosys step for adding clean designs also lives here.

Wrap-up

- **P10:** packaging, run_all.sh, and the final report.

Your PDF’s tech stack also lists DGL and PyTorch Lightning. The plan uses neither: PyG alone, with the custom Trainer from 3A in place of Lightning. If your project needs Lightning for a reason I’m not seeing, it’s an easy swap in 3A.

well can you also explain in simple words of what I am trying to build

You're building a system that checks a chip design for a hidden backdoor and then explains what it found.

The problem

A chip is made from thousands of tiny logic gates wired together. A hardware Trojan is a small group of extra gates that someone sneaks into the design. Most of the time it does nothing, so normal testing never notices it. When a rare, secret condition happens, it kicks in and does damage, like leaking data or breaking the chip. Finding those few gates among thousands is hard.

Your solution has two brains

1. **The GNN (the detector).** You turn the design into a graph, where each gate is a dot and each wire is a line between dots. For every gate you record things like what type it is, how many wires connect to it, and how rarely it switches on. The GNN learns what Trojan gates tend to look like and gives every gate a suspicion score. It's fast, but it only says "this gate looks suspicious." It can't say why.
2. **The LLM (the explainer).** You take the most suspicious gates plus the gates around them and describe them to a language model in plain structured text. You ask

whether this looks like a Trojan trigger, why, and how to fix it. It answers with a verdict (Trojan, benign or uncertain), an explanation a person can read, and a fix suggestion.

The extra safety checks

LLMs can be unreliable. They can change their answer based on the order you show things, or just guess. So you ask the same question in different orders and to different models, and you only trust answers that agree. That is your bias and consistency control.

The feedback loop

Answers from the LLM that you trust are fed back to retrain the GNN. Over time the GNN gets better and you need the slow LLM less often.

The one-line version

A fast detector finds suspicious gates, a language model explains and checks them, and the explanations teach the detector to improve.

What makes your version different from your friend's

Your friend's project uses only one GNN and only the wiring structure, and its LLM step is a fixed one-way handoff. You add several GNNs that look at the design from different angles, behavior features from simulation, stricter checks on the LLM, and the feedback loop. The plan treats each of those as something to test, not assume, because your friend's own results showed the LLM didn't beat a simple "pick the highest GNN score" rule. If your extras measurably beat that, you have a real result. If they don't, you still have an honest, well-measured negative result, which is also worth reporting.